Digital Signal Processing 20 (2010) 1458–1467

Contents lists available at ScienceDirect

Digital Signal Processing

www.elsevier.com/locate/dsp

Least squares based iterative algorithms for identifying Box–Jenkins models with finite measurement data ☆

Yanjun Liu [a], Dongqing Wang [b], Feng Ding [a,*]

[a] School of Communication and Control Engineering, Jiangnan University, Wuxi, PR China 214122
[b] College of Automation Engineering, Qingdao University, Qingdao, PR China 266071

ARTICLE INFO

Article history:
Available online 13 January 2010

Keywords:
Jacobi iterations
Gauss–Seidel iterations
Recursive identification
Parameter estimation
Least squares
Box–Jenkins models

ABSTRACT

A least squares based iterative identification algorithm is developed for Box–Jenkins models (or systems). The proposed iterative algorithm can produce highly accurate parameter estimation compared with recursive approaches. The basic idea of the proposed iterative method is to adopt the interactive estimation theory: the parameter estimates relying on unknown variables are computed by using the estimates of these unknown variables which are obtained from the preceding parameter estimates. The numerical example indicates that the proposed iterative algorithm has fast convergence rates compared with the gradient based iterative algorithm.

© 2010 Elsevier Inc. All rights reserved.

1. Introduction

Iterative methods are very important in finding the roots of nonlinear equations and the minimum/maximum values of functions and in solving a set of linear systems, e.g., the Jacobi iteration and Gauss–Seidel iterations [1]. The gradient based and least squares based iterative methods are proposed for matrix equations $\boldsymbol{AX}+\boldsymbol{XB}=\boldsymbol{F}$ [2,3] and coupled matrix equations $\boldsymbol{AX}+\boldsymbol{YB}=\boldsymbol{F}_1$ and $\boldsymbol{CX}+\boldsymbol{YD}=\boldsymbol{F}_2$ [4], introducing the star ($\star$) product of the block matrices [4]. Based on these results, Kilicman and Zhour discussed the weighted least squares iterative algorithms for such matrix equations [5] and Zhour and Kilicman studied the operation relation of partitioned matrices [6].

In the area of system identification and parameter estimation, iterative methods are often used to estimate the parameters of equations or models from the measured data [7,8]. Recently, Ding, Shi and Chen used iterative methods to estimate the parameters c_i and d_i of the following autoregressive moving average (ARMA) model [9]:

$$C(z)y(t)=D(z)v(t)$$

or

$$y(t)+c_1y(t-1)+\cdots+c_{n_c}y(t-n_c)=v(t)+d_1v(t-1)+\cdots+d_{n_d}v(t-n_d)$$

only using the observations $\{y(t)\colon\ t=1,2,\ldots,L\}$ (L represents the data length), where c_i and d_j are the parameters of the ARMA models and the definitions of other variables are given in the next section.

The so-called system identification or parameter estimation is to present (recursive/iterative) algorithms to compute/estimate the system parameters from the available observation data. This paper considers identification problems of the following Box–Jenkins models or systems, depicted in Fig. 1,

☆ This work was supported by the National Natural Science Foundation of China (60973043).
* Corresponding author.
E-mail addresses: yanjunliu_1983@126.com (Y.J. Liu), dqwang64@163.com (D.Q. Wang), fding@jiangnan.edu.cn (F. Ding).

1051-2004/$ – see front matter © 2010 Elsevier Inc. All rights reserved.
doi:10.1016/j.dsp.2010.01.004

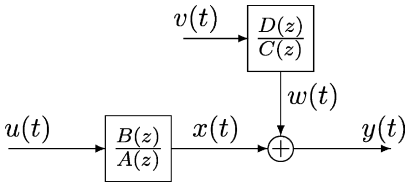


Fig. 1. A system described by the Box–Jenkins model.

$$y(t) = \frac{B(z)}{A(z)}u(t) + \frac{D(z)}{C(z)}v(t), \tag{1}$$

where $\{u(t)\}$ and $\{y(t)\}$ are the input and output sequences of the system, respectively, $\{v(t)\}$ is a white noise sequence with zero mean, and $A(z)$, $B(z)$, $C(z)$ and $D(z)$ are the polynomials, of known orders (n_a, n_b, n_c, n_d), in the unit backward shift operator z^{-1} [i.e., $z^{-1}y(t) = y(t-1)$], defined by

$$\begin{aligned}
A(z) &= 1 + a_1 z^{-1} + a_2 z^{-2} + \cdots + a_{n_a} z^{-n_a},\\
B(z) &= b_1 z^{-1} + b_2 z^{-2} + \cdots + b_{n_b} z^{-n_b},\\
C(z) &= 1 + c_1 z^{-1} + c_2 z^{-2} + \cdots + c_{n_c} z^{-n_c},\\
D(z) &= 1 + d_1 z^{-1} + d_2 z^{-2} + \cdots + d_{n_d} z^{-n_d}.
\end{aligned}$$

The objective of this paper is to develop least squares based iterative algorithms to estimate the unknown parameters (a_i, b_i, c_i, d_i) by using the available input–output measurement data $\{u(t), y(t)\colon\ t = 1, 2, \ldots, L\}$ to improve the accuracy of parameter estimation.

The iterative methods are generally used for the parameter identification for nonlinear systems [10–12] and lead to two-stage identification algorithms which estimate the parameters of the linear parts and nonlinear parts, respectively. On the basis of the iterative algorithm for output error (OE) and output error moving average (OEMA) models in [13], this paper applies the iterative method to identify the Box–Jenkins models and the basic idea is to replace the unmeasurable noise terms in the information vector with their iterative estimates. To the best of our knowledge, the traditional approach of identifying the Box–Jenkins models is by minimizing the prediction error criterion functions and the resulting methods are the prediction error based recursive least squares algorithm [8] (see Section 2) and prediction error stochastic gradient algorithm [14]. The merit of the iterative algorithm is that it can produce more accurate parameter estimates than the corresponding recursive algorithms.

2. The recursive algorithm for Box–Jenkins systems

In order to show the advantages of the iterative algorithm we will propose later, we give the recursive algorithms for the Box–Jenkins models.

Referring to Fig. 1, define the (unknown) middle variables,

$$x(t) := \frac{B(z)}{A(z)}u(t), \qquad w(t) := \frac{D(z)}{C(z)}v(t). \tag{2}$$

From (1), we have

$$y(t) = x(t) + w(t). \tag{3}$$

Note that $x(t)$ represents the true output (i.e., noise-free output) of the system, $y(t)$ is the measurement of $x(t)$ and is corrupted by the colored noise $w(t)$ which is regarded as the output of the ARMA noise model $\frac{D(z)}{C(z)}$ driven by the white noise $v(t)$. The inner variables $x(t)$ and $w(t)$ are unmeasurable.

Define the parameter vectors

$$\begin{aligned}
\boldsymbol{\theta} &:= \begin{bmatrix} \boldsymbol{\theta}_{\mathrm{s}} \\ \boldsymbol{\theta}_{\mathrm{n}} \end{bmatrix} \in \mathbb{R}^{n_a+n_b+n_c+n_d},\\
\boldsymbol{\theta}_{\mathrm{s}} &:= [a_1, a_2, \ldots, a_{n_a}, b_1, b_2, \ldots, b_{n_b}]^{\mathrm{T}} \in \mathbb{R}^{n_a+n_b},\\
\boldsymbol{\theta}_{\mathrm{n}} &:= [c_1, c_2, \ldots, c_{n_c}, d_1, d_2, \ldots, d_{n_d}]^{\mathrm{T}} \in \mathbb{R}^{n_c+n_d},
\end{aligned}$$

and the information vectors

$$\begin{aligned}
\boldsymbol{\varphi}(t) &:= \begin{bmatrix} \boldsymbol{\varphi}_{\mathrm{s}}(t) \\ \boldsymbol{\varphi}_{\mathrm{n}}(t) \end{bmatrix} \in \mathbb{R}^{n_a+n_b+n_c+n_d},\\
\boldsymbol{\varphi}_{\mathrm{s}}(t) &:= \big[-x(t-1), -x(t-2), \ldots, -x(t-n_a), u(t-1), u(t-2), \ldots, u(t-n_b)\big]^{\mathrm{T}} \in \mathbb{R}^{n_a+n_b},\\
\boldsymbol{\varphi}_{\mathrm{n}}(t) &:= \big[-w(t-1), -w(t-2), \ldots, -w(t-n_c), v(t-1), v(t-2), \ldots, v(t-n_d)\big]^{\mathrm{T}} \in \mathbb{R}^{n_c+n_d},
\end{aligned}$$

where subscripts (roman) s and n denote the first letters of the words 'system' and 'noise', respectively. Eqs. (2)–(3) can be written as

$$x(t) = \boldsymbol{\varphi}_{\mathrm{s}}^{\mathrm{T}}(t)\boldsymbol{\theta}_{\mathrm{s}}, \tag{4}$$

$$w(t) = \boldsymbol{\varphi}_{\mathrm{n}}^{\mathrm{T}}(t)\boldsymbol{\theta}_{\mathrm{n}} + v(t), \tag{5}$$

$$y(t) = \boldsymbol{\varphi}_{\mathrm{s}}^{\mathrm{T}}(t)\boldsymbol{\theta}_{\mathrm{s}} + \boldsymbol{\varphi}_{\mathrm{n}}^{\mathrm{T}}(t)\boldsymbol{\theta}_{\mathrm{n}} + v(t) = \boldsymbol{\varphi}^{\mathrm{T}}(t)\boldsymbol{\theta} + v(t). \tag{6}$$

Eq. (6) is the pseudo-linear regressive identification model for the Box–Jenkins systems and the parameter vector $\boldsymbol{\theta}$ contains all the parameters (a_i, b_i, c_i, d_i) of the systems. Difficulty of identification is that $x(t-i)$, $w(t-i)$ and $v(t-i)$ in the information vector $\boldsymbol{\varphi}(t)$ are unknown. If these unknown variables $x(t-i)$, $w(t-i)$ and $v(t-i)$ are replaced with their estimates $\hat{x}(t-i)$, $\hat{w}(t-i)$ and $\hat{v}(t-i)$, then minimizing the cost function

$$J(\boldsymbol{\theta}) := \sum_{i=1}^{t}\big[y(i) - \hat{\boldsymbol{\varphi}}^{\mathrm{T}}(i)\boldsymbol{\theta}\big]^2,$$

one can obtain the following recursive generalized extended least squares algorithm of estimating $\boldsymbol{\theta}$ for the Box–Jenkins systems (BJ-RGELS algorithm for short):

$$\hat{\boldsymbol{\theta}}(t) = \hat{\boldsymbol{\theta}}(t-1) + \boldsymbol{P}(t)\hat{\boldsymbol{\varphi}}(t)\big[y(t) - \hat{\boldsymbol{\varphi}}^{\mathrm{T}}(t)\hat{\boldsymbol{\theta}}(t-1)\big], \tag{7}$$

$$\boldsymbol{P}(t) = \boldsymbol{P}(t-1) - \frac{\boldsymbol{P}(t-1)\hat{\boldsymbol{\varphi}}(t)\hat{\boldsymbol{\varphi}}^{\mathrm{T}}(t)\boldsymbol{P}(t-1)}{1 + \hat{\boldsymbol{\varphi}}^{\mathrm{T}}(t)\boldsymbol{P}(t-1)\hat{\boldsymbol{\varphi}}(t)}, \qquad \boldsymbol{P}(0) = p_0\boldsymbol{I}, \tag{8}$$

$$\hat{\boldsymbol{\varphi}}(t) = \begin{bmatrix} \hat{\boldsymbol{\varphi}}_{\mathrm{s}}(t) \\ \hat{\boldsymbol{\varphi}}_{\mathrm{n}}(t) \end{bmatrix}, \qquad \hat{\boldsymbol{\theta}}(t) = \begin{bmatrix} \hat{\boldsymbol{\theta}}_{\mathrm{s}}(t) \\ \hat{\boldsymbol{\theta}}_{\mathrm{n}}(t) \end{bmatrix}, \tag{9}$$

$$\hat{\boldsymbol{\varphi}}_{\mathrm{s}}(t) := \big[-\hat{x}(t-1), -\hat{x}(t-2), \ldots, -\hat{x}(t-n_a), u(t-1), u(t-2), \ldots, u(t-n_b)\big]^{\mathrm{T}}, \tag{10}$$

$$\hat{\boldsymbol{\varphi}}_{\mathrm{n}}(t) := \big[-\hat{w}(t-1), -\hat{w}(t-2), \ldots, -\hat{w}(t-n_c), \hat{v}(t-1), \hat{v}(t-2), \ldots, \hat{v}(t-n_d)\big]^{\mathrm{T}}, \tag{11}$$

$$\hat{x}(t) = \hat{\boldsymbol{\varphi}}_{\mathrm{s}}^{\mathrm{T}}(t)\hat{\boldsymbol{\theta}}_{\mathrm{s}}(t), \tag{12}$$

$$\hat{w}(t) = y(t) - \hat{x}(t), \tag{13}$$

$$\hat{v}(t) = \hat{w}(t) - \hat{\boldsymbol{\varphi}}_{\mathrm{n}}^{\mathrm{T}}(t)\hat{\boldsymbol{\theta}}_{\mathrm{n}}(t), \tag{14}$$

$\hat{\boldsymbol{\theta}}(t) = \left[\begin{smallmatrix} \hat{\boldsymbol{\theta}}_{\mathrm{s}}(t) \\ \hat{\boldsymbol{\theta}}_{\mathrm{n}}(t) \end{smallmatrix}\right]$ is the estimate of $\boldsymbol{\theta}$. The BJ-RGELS algorithm is also called the prediction error least squares one which can be derived according to the prediction error criterion [8].

To initialize the algorithm, we take $\hat{\boldsymbol{\theta}}(0)$ to be some small real vector, e.g., $\hat{\boldsymbol{\theta}}(0) = \mathbf{1}_n/p_0$ with $\mathbf{1}_n$ being an $n := (n_a + n_b + n_c + n_d)$-dimensional column vector whose elements are all 1 and with p_0 being normally a large positive number (e.g., $p_0 = 10^6$), and $\boldsymbol{P}(0) = p_0\boldsymbol{I}$ with $\boldsymbol{I}$ representing an identity matrix of appropriate sizes.

To summarize, we list the steps involved in the BJ-RGELS algorithm to compute $\hat{\boldsymbol{\theta}}(L)$:

1. Collect the input–output data $\{u(t), y(t)\colon\ t = 1, 2, \ldots, L\}$ (L is the data length) and assume that $u(i) = 0$ for $i \leqslant 0$.
2. To initialize, let $t = 1$, $\hat{\boldsymbol{\theta}}(0) = \mathbf{1}_n/p_0$, $\boldsymbol{P}(0) = p_0\boldsymbol{I}$, $\hat{x}(i) = 1/p_0$, $\hat{w}(i) = 1/p_0$ and $\hat{v}(i) = 1/p_0$ for $i \leqslant 0$ and $p_0 = 10^6$.
3. Form $\hat{\boldsymbol{\varphi}}_{\mathrm{s}}(t)$ by (10), $\hat{\boldsymbol{\varphi}}_{\mathrm{n}}(t)$ by (11) and $\hat{\boldsymbol{\varphi}}(t)$ by (9).
4. Compute the covariance matrix $\boldsymbol{P}(t)$ by (8).
5. Update the parameter estimation vector $\hat{\boldsymbol{\theta}}(t)$ by (7).
6. Compute $\hat{x}(t)$ by (12), $\hat{w}(t)$ by (13) and $\hat{v}(t)$ by (14).
7. If $t < L$, increase t by 1 and go to step 3; otherwise, obtain the parameter estimation vector $\hat{\boldsymbol{\theta}}(L)$.

The flowchart of computing the parameter estimate $\hat{\boldsymbol{\theta}}(t)$ is shown in Fig. 2.

3. The least squares based iterative algorithm

Next, we investigate the least squares based iterative identification approach for the Box–Jenkins systems with finite measurement input–output data $\{u(t), y(t)\colon\ t = 0, 1, 2, \ldots, L\}$.

Consider the newest p data and define the stacked output vector $\boldsymbol{Y}(t)$ and stacked information matrix $\boldsymbol{\Phi}(t)$ as

$$\boldsymbol{Y}(t) = \begin{bmatrix} y(t) \\ y(t-1) \\ \vdots \\ y(t-p+1) \end{bmatrix} \in \mathbb{R}^p, \tag{15}$$

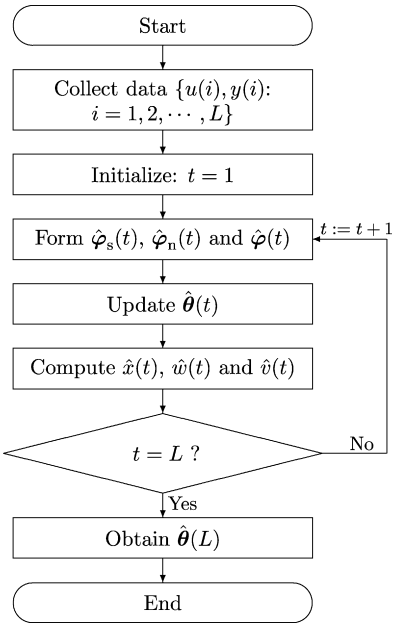


Fig. 2. The flowchart of computing the recursive estimate $\hat{\boldsymbol{\theta}}(L)$.

$$\boldsymbol{\Phi}(t) = \begin{bmatrix} \boldsymbol{\varphi}^{\mathrm{T}}(t) \\ \boldsymbol{\varphi}^{\mathrm{T}}(t-1) \\ \vdots \\ \boldsymbol{\varphi}^{\mathrm{T}}(t-p+1) \end{bmatrix} \in \mathbb{R}^{p\times n}. \tag{16}$$

From (6), (15) and (16), define a quadratic criterion function [8],

$$J(\boldsymbol{\theta}) = \big\|\boldsymbol{Y}(t) - \boldsymbol{\Phi}(t)\boldsymbol{\theta}\big\|^2, \qquad \|\boldsymbol{X}\|^2 := \mathrm{tr}\big[\boldsymbol{X}\boldsymbol{X}^{\mathrm{T}}\big]. \tag{17}$$

For the optimization problem in (17), provided that $\boldsymbol{\varphi}(t)$ is persistently exciting, minimizing $J(\boldsymbol{\theta})$ in (17) gives the least squares estimate of $\boldsymbol{\theta}$:

$$\hat{\boldsymbol{\theta}}(t) = \big[\boldsymbol{\Phi}^{\mathrm{T}}(t)\boldsymbol{\Phi}(t)\big]^{-1}\boldsymbol{\Phi}^{\mathrm{T}}(t)\boldsymbol{Y}(t). \tag{18}$$

Here, a difficulty arises because $\boldsymbol{\Phi}(t)$ in (18) [i.e., $\boldsymbol{\varphi}(t)$] contains unknown inner variables $x(t-i)$, $w(t-i)$ and noise terms $v(t-i)$, so it is impossible to compute the solution $\hat{\boldsymbol{\theta}}(t)$ of $\boldsymbol{\theta}$ by (18). The solution here is based on the hierarchical identification principle [2–4].

The iterative algorithm employs the idea of updating the estimate $\hat{\boldsymbol{\theta}}$ using the fixed length data. In this paper, in order to distinguish on-line from off-line calculation, we use *iterative* with subscript k, e.g., $\hat{\boldsymbol{\theta}}_k$ to denote the parameter estimate for off-line algorithms, and *recursive* with no subscript, e.g., $\hat{\boldsymbol{\theta}}(t)$ before to denote the parameter estimate for on-line ones.

Let $k = 1, 2, 3, \ldots$ be an iteration variable, and $\hat{x}_k(t-i)$, $\hat{w}_k(t-i)$ and $\hat{v}_k(t-i)$ be the estimates of $x(t-i)$, $w(t-i)$ and $v(t-i)$ at iteration k, and $\hat{\boldsymbol{\varphi}}_k(t)$ denote the information vector obtained by replacing $x(t-i)$, $w(t-i)$ and $v(t-i)$ in $\boldsymbol{\varphi}(t)$ with $\hat{x}_k(t-i)$, $\hat{w}_k(t-i)$ and $\hat{v}_k(t-i)$, i.e.,

$$\hat{\boldsymbol{\varphi}}_k(t) := \begin{bmatrix} \hat{\boldsymbol{\varphi}}_{s,k}(t) \\ \hat{\boldsymbol{\varphi}}_{n,k}(t) \end{bmatrix} \in \mathbb{R}^{n_a+n_b+n_c+n_d}$$

with

$$\hat{\boldsymbol{\varphi}}_{s,k}(t) := \big[-\hat{x}_{k-1}(t-1), -\hat{x}_{k-1}(t-2), \ldots, -\hat{x}_{k-1}(t-n_a), u(t-1), u(t-2), \ldots, u(t-n_b)\big]^{\mathrm{T}} \in \mathbb{R}^{n_a+n_b},$$

$$\hat{\boldsymbol{\varphi}}_{n,k}(t) := \big[-\hat{w}_{k-1}(t-1), -\hat{w}_{k-1}(t-2), \ldots, -\hat{w}_{k-1}(t-n_c), \hat{v}_{k-1}(t-1), \hat{v}_{k-1}(t-2), \ldots, \hat{v}_{k-1}(t-n_d)\big]^{\mathrm{T}} \in \mathbb{R}^{n_c+n_d}.$$

Let $\hat{\boldsymbol{\theta}}_k(t) := \left[\begin{smallmatrix} \hat{\boldsymbol{\theta}}_{s,k}(t) \\ \hat{\boldsymbol{\theta}}_{n,k}(t) \end{smallmatrix}\right]$ be the estimate of $\boldsymbol{\theta} = \left[\begin{smallmatrix} \boldsymbol{\theta}_s \\ \boldsymbol{\theta}_n \end{smallmatrix}\right]$ at iteration k. From (4), we have

$$x(t-i) = \boldsymbol{\varphi}_s^{\mathrm{T}}(t-i)\boldsymbol{\theta}_s.$$

Replacing $\boldsymbol{\varphi}_s(t-i)$ and $\boldsymbol{\theta}_s$ in the above equation with their estimates $\hat{\boldsymbol{\varphi}}_{s,k}(t-i)$ and $\hat{\boldsymbol{\theta}}_{s,k}(t)$, respectively, we can compute the estimate $\hat{x}_k(t-i)$ by

$$\hat{x}_k(t-i) = \hat{\boldsymbol{\varphi}}_{\mathrm{s},k}^{\mathrm{T}}(t-i)\hat{\boldsymbol{\theta}}_{\mathrm{s},k}(t).$$

From (3), we have

$$w(t-i) = y(t-i) - x(t-i).$$

Then the estimate of $w(t-i)$ can be computed by

$$\hat{w}_k(t-i) = y(t-i) - \hat{x}_k(t-i) = y(t-i) - \hat{\boldsymbol{\varphi}}_{\mathrm{s},k}^{\mathrm{T}}(t-i)\hat{\boldsymbol{\theta}}_{\mathrm{s},k}(t).$$

From (6), we have

$$v(t-i) = y(t-i) - \boldsymbol{\varphi}^{\mathrm{T}}(t-i)\boldsymbol{\theta}.$$

If $\boldsymbol{\varphi}(t-i)$ and $\boldsymbol{\theta}$ in the above equation are replaced with their estimates $\hat{\boldsymbol{\varphi}}_{k-1}(t-i)$ and $\hat{\boldsymbol{\theta}}_{k-1}(t)$, then the estimate of $v(t-i)$ can be computed by

$$\hat{v}_k(t-i) = y(t-i) - \hat{\boldsymbol{\varphi}}_k^{\mathrm{T}}(t-i)\hat{\boldsymbol{\theta}}_k(t).$$

Of course, from (5), we have

$$v(t-i) = w(t-i) - \boldsymbol{\varphi}_{\mathrm{n}}^{\mathrm{T}}(t-i)\boldsymbol{\theta}_{\mathrm{n}}.$$

Thus, the estimate $\hat{v}_k(t-i)$ can also be computed by

$$\hat{v}_k(t-i) = \hat{w}_k(t-i) - \hat{\boldsymbol{\varphi}}_{\mathrm{n},k}^{\mathrm{T}}(t-i)\hat{\boldsymbol{\theta}}_{\mathrm{n},k}(t).$$

Define

$$\hat{\boldsymbol{\Phi}}_k(t) = \begin{bmatrix} \hat{\boldsymbol{\varphi}}_k^{\mathrm{T}}(t) \\ \hat{\boldsymbol{\varphi}}_k^{\mathrm{T}}(t-1) \\ \vdots \\ \hat{\boldsymbol{\varphi}}_k^{\mathrm{T}}(t-p+1) \end{bmatrix} \in \mathbb{R}^{p\times n}.$$

Replacing $\boldsymbol{\Phi}(t)$ in (18) with $\hat{\boldsymbol{\Phi}}_k(t)$ yields the following least squares based identification algorithm for estimating $\boldsymbol{\theta}$:

$$\hat{\boldsymbol{\theta}}_k(t) = \big[\hat{\boldsymbol{\Phi}}_k^{\mathrm{T}}(t)\hat{\boldsymbol{\Phi}}_k(t)\big]^{-1}\hat{\boldsymbol{\Phi}}_k^{\mathrm{T}}(t)\boldsymbol{Y}(t), \tag{19}$$

$$\hat{\boldsymbol{\Phi}}_k(t) = \big[\hat{\boldsymbol{\varphi}}_k(t), \hat{\boldsymbol{\varphi}}_k(t-1), \ldots, \hat{\boldsymbol{\varphi}}_k(t-p+1)\big]^{\mathrm{T}}, \tag{20}$$

$$\boldsymbol{Y}(t) = \big[y(t), y(t-1), \ldots, y(t-p+1)\big]^{\mathrm{T}}, \tag{21}$$

$$\hat{\boldsymbol{\varphi}}_k(t) = \begin{bmatrix} \hat{\boldsymbol{\varphi}}_{\mathrm{s},k}(t) \\ \hat{\boldsymbol{\varphi}}_{\mathrm{n},k}(t) \end{bmatrix}, \tag{22}$$

$$\hat{\boldsymbol{\varphi}}_{\mathrm{s},k}(t) = \big[-\hat{x}_{k-1}(t-1), -\hat{x}_{k-1}(t-2), \ldots, -\hat{x}_{k-1}(t-n_a), u(t-1), u(t-2), \ldots, u(t-n_b)\big]^{\mathrm{T}}, \tag{23}$$

$$\hat{\boldsymbol{\varphi}}_{\mathrm{n},k}(t) = \big[-\hat{w}_{k-1}(t-1), -\hat{w}_{k-1}(t-2), \ldots, -\hat{w}_{k-1}(t-n_c), \hat{v}_{k-1}(t-1), \hat{v}_{k-1}(t-2), \ldots, \hat{v}_{k-1}(t-n_d)\big]^{\mathrm{T}}, \tag{24}$$

$$\hat{\boldsymbol{\theta}}_k(t) = \begin{bmatrix} \hat{\boldsymbol{\theta}}_{\mathrm{s},k}(t) \\ \hat{\boldsymbol{\theta}}_{\mathrm{n},k}(t) \end{bmatrix}, \tag{25}$$

$$\hat{x}_k(t-i) = \hat{\boldsymbol{\varphi}}_{\mathrm{s},k}^{\mathrm{T}}(t-i)\hat{\boldsymbol{\theta}}_{\mathrm{s},k}(t), \tag{26}$$

$$\hat{w}_k(t-i) = y(t-i) - \hat{x}_k(t-i) = y(t-i) - \hat{\boldsymbol{\varphi}}_{\mathrm{s},k}^{\mathrm{T}}(t-i)\hat{\boldsymbol{\theta}}_{\mathrm{s},k}(t), \tag{27}$$

$$\hat{v}_k(t-i) = \hat{w}_k(t-i) - \hat{\boldsymbol{\varphi}}_{\mathrm{n},k}^{\mathrm{T}}(t-i)\hat{\boldsymbol{\theta}}_{\mathrm{n},k}(t) = y(t-i) - \hat{\boldsymbol{\varphi}}_k^{\mathrm{T}}(t-i)\hat{\boldsymbol{\theta}}_k(t). \tag{28}$$

Eqs. (19)–(28) are referred to as the least squares based iterative identification algorithm for Box–Jenkins systems, which is abbreviated as the BJ-LSI algorithm. The BJ-LSI algorithm adopts the idea of updating the estimate $\hat{\boldsymbol{\theta}}$ using a fixed data batch with the data length p at each iteration, and thus has higher parameter estimation accuracy than the BJ-RGELS algorithm.

The steps of implementing the BJ-LSI algorithm to compute $\hat{\boldsymbol{\theta}}_k(t)$ as k and t increases are as follows:

1. Given p and iterative time k_0 (e.g., $k_0 = 6$), let $t = p$ and collect the input–output data $\{u(i), y(i)\colon\ i = 1, 2, \ldots, p-1\}$.
2. To initialize, let $k = 1$, $\hat{x}_0(t-i) =$ random number, $\hat{w}_0(t-i) =$ random number and $\hat{v}_0(t-i) =$ random number.
3. Collect the input–output data $\{u(t), y(t)\}$ and form $\boldsymbol{Y}(t)$ by (21).
4. Form $\hat{\boldsymbol{\varphi}}_{\mathrm{s},k}(t)$, $\hat{\boldsymbol{\varphi}}_{\mathrm{n},k}(t)$ and $\hat{\boldsymbol{\varphi}}_k(t)$ by (23), (24) and (22) and $\hat{\boldsymbol{\Phi}}_k(t)$ by (20).
5. Update the estimate $\hat{\boldsymbol{\theta}}_k(t)$ by (19).

6. Compute $\hat{x}_k(t-i)$ by (26), $\hat{w}_k(t-i)$ by (27) and $\hat{v}_k(t-i)$ by (28).
7. If $k < k_0$,
   increase k by 1 and go to step 4,
   else
   obtain the parameter estimate $\hat{\boldsymbol{\theta}}_{k_0}(t)$ at time t,
   set $\hat{\boldsymbol{\theta}}_0(t+1) = \hat{\boldsymbol{\theta}}_{k_0}(t)$ and $k = 1$,
   increase t by 1 and go to step 3,
   end.

Since the BJ-LSI algorithm has very fast convergence rates compared with the gradient based iterative algorithm in [14], we can obtain highly accurate parameter estimate after several iterations – see the example later. This is why we take $k_0 = 6$ above.

The iterative algorithms are also suitable for on-line identification and can improve estimation accuracy, compared with the recursive algorithms which can be used for on-line identification. Although the iterative algorithms require large computational efforts, this increased computation is still tolerable and affordable for modern computers.

Here, if we set $p = L$ and $t = L$ in (15)–(16) (L: the data length), then

$$\boldsymbol{Y}(L) = \begin{bmatrix} y(L) \\ y(L-1) \\ \vdots \\ y(1) \end{bmatrix} \in \mathbb{R}^L, \qquad \boldsymbol{\Phi}(L) = \begin{bmatrix} \boldsymbol{\varphi}^{\mathrm{T}}(L) \\ \boldsymbol{\varphi}^{\mathrm{T}}(L-1) \\ \vdots \\ \boldsymbol{\varphi}^{\mathrm{T}}(1) \end{bmatrix} \in \mathbb{R}^{L\times n}$$

contain all the measured data available $\{u(i), y(i)\colon\ i = 1, 2, \ldots, L\}$. Similarly, defining and minimizing the quadratic criterion function [8],

$$J_1(\boldsymbol{\theta}) = \left\| \boldsymbol{Y}(L) - \boldsymbol{\Phi}(L)\boldsymbol{\theta} \right\|^2, \tag{29}$$

yield the following BJ-LSI algorithm with finite measurement data:

$$\hat{\boldsymbol{\theta}}_k(L) = \left[\hat{\boldsymbol{\Phi}}_k^{\mathrm{T}}(L)\hat{\boldsymbol{\Phi}}_k(L)\right]^{-1}\hat{\boldsymbol{\Phi}}_k^{\mathrm{T}}(L)\boldsymbol{Y}(L), \tag{30}$$

$$\hat{\boldsymbol{\Phi}}_k(L) = \left[\hat{\boldsymbol{\varphi}}_k(L), \hat{\boldsymbol{\varphi}}_k(L-1), \ldots, \hat{\boldsymbol{\varphi}}_k(1)\right]^{\mathrm{T}}, \tag{31}$$

$$\boldsymbol{Y}(L) = \left[y(L), y(L-1), \ldots, y(1)\right]^{\mathrm{T}}, \tag{32}$$

$$\hat{\boldsymbol{\varphi}}_k(t) = \begin{bmatrix} \hat{\boldsymbol{\varphi}}_{\mathrm{s},k}(t) \\ \hat{\boldsymbol{\varphi}}_{\mathrm{n},k}(t) \end{bmatrix}, \quad t = 1, 2, \ldots, L, \tag{33}$$

$$\hat{\boldsymbol{\varphi}}_{\mathrm{s},k}(t) = \left[-\hat{x}_{k-1}(t-1), -\hat{x}_{k-1}(t-2), \ldots, -\hat{x}_{k-1}(t-n_a), u(t-1), u(t-2), \ldots, u(t-n_b)\right]^{\mathrm{T}}, \tag{34}$$

$$\hat{\boldsymbol{\varphi}}_{\mathrm{n},k}(t) = \left[-\hat{w}_{k-1}(t-1), -\hat{w}_{k-1}(t-2), \ldots, -\hat{w}_{k-1}(t-n_c), \hat{v}_{k-1}(t-1), \hat{v}_{k-1}(t-2), \ldots, \hat{v}_{k-1}(t-n_d)\right]^{\mathrm{T}}, \tag{35}$$

$$\hat{\boldsymbol{\theta}}_k(L) = \begin{bmatrix} \hat{\boldsymbol{\theta}}_{\mathrm{s},k}(L) \\ \hat{\boldsymbol{\theta}}_{\mathrm{n},k}(L) \end{bmatrix}, \tag{36}$$

$$\hat{x}_k(t) = \hat{\boldsymbol{\varphi}}_{\mathrm{s},k}^{\mathrm{T}}(t)\hat{\boldsymbol{\theta}}_{\mathrm{s},k}(L), \tag{37}$$

$$\hat{w}_k(t) = y(t) - \hat{x}_k(t) = y(t) - \hat{\boldsymbol{\varphi}}_{\mathrm{s},k}^{\mathrm{T}}(t)\hat{\boldsymbol{\theta}}_{\mathrm{s},k}(L), \tag{38}$$

$$\hat{v}_k(t) = \hat{w}_k(t) - \hat{\boldsymbol{\varphi}}_{\mathrm{n},k}^{\mathrm{T}}(t)\hat{\boldsymbol{\theta}}_{\mathrm{n},k}(L) = y(t) - \hat{\boldsymbol{\varphi}}_k^{\mathrm{T}}(t)\hat{\boldsymbol{\theta}}_k(L). \tag{39}$$

To summarize, we list the steps involved in the BJ-LSI algorithm to compute $\hat{\boldsymbol{\theta}}_k(L)$ as k increases:

1. Collect the input–output data $\{u(i), y(i)\colon\ i = 1, 2, \ldots, L\}$, form $\boldsymbol{Y}(t)$ by (32).
2. To initialize, let $k = 1$, $\hat{x}_0(t) =$ random number, $\hat{w}_0(t) =$ random number and $\hat{v}_0(t) =$ random number.
3. Form $\hat{\boldsymbol{\varphi}}_{\mathrm{s},k}(t)$ by (34), $\hat{\boldsymbol{\varphi}}_{\mathrm{n},k}(t)$ by (35), $\hat{\boldsymbol{\varphi}}_k(t)$ by (33) and $\hat{\boldsymbol{\Phi}}_k(L)$ by (31).
4. Update the estimate $\hat{\boldsymbol{\theta}}_k(L)$ by (30).
5. Compute $\hat{x}_k(t)$ by (37), $\hat{w}_k(t)$ by (38) and $\hat{v}_k(t)$ by (39).
6. Compare $\hat{\boldsymbol{\theta}}_k(L)$ with $\hat{\boldsymbol{\theta}}_{k-1}(L)$: if they are sufficiently close, or for some pre-set small ε, if

$$\left\| \hat{\boldsymbol{\theta}}_k(L) - \hat{\boldsymbol{\theta}}_{k-1}(L) \right\| \leqslant \varepsilon,$$

   then terminate the procedure and obtain the iterative times k and estimate $\hat{\boldsymbol{\theta}}_k(L)$; otherwise, increase k by 1 and go to step 3.

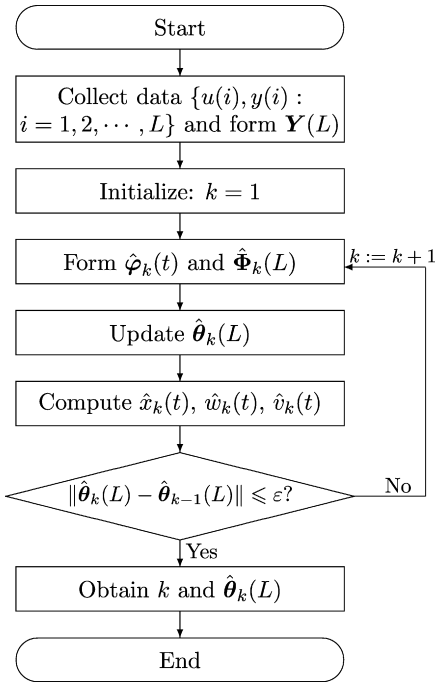


Fig. 3. The flowchart of computing the iterative estimate $\hat{\boldsymbol{\theta}}_k(L)$.

The flowchart of computing the parameter estimate $\hat{\boldsymbol{\theta}}_k(L)$ is shown in Fig. 3.

This BJ-LSI iterative algorithm in (30)–(39) is suitable only for off-line identification and can improve estimation accuracy with recursive algorithm in (7)–(14) under the same data length. The iterative algorithm is generally used for finite measurements, e.g., the algorithm in (30)–(39) requires that the data length L is finite.

In general, the least squares algorithm has faster convergence rates than the stochastic gradient algorithm [7,8]. Similarly, the least squares based iterative algorithm has faster convergence rates than the gradient based iterative algorithm. The convergence of the BJ-LSI algorithm may be proved by using a similar method to one in [11,12].

4. Example

Consider the Box–Jenkins system,

$$\begin{cases} y(t) = \dfrac{B(z)}{A(z)}\,u(t) + \dfrac{D(z)}{C(z)}\,v(t), \\ A(z) = 1 + a_1 z^{-1} + a_2 z^{-2} = 1 - 0.40z^{-1} + 0.44z^{-2}, \\ B(z) = b_1 z^{-1} + b_2 z^{-2} = -1.80z^{-1} + 0.24z^{-2}, \\ C(z) = 1 + c_1 z^{-1} = 1 - 0.48z^{-1}, \\ D(z) = 1 + d_1 z^{-1} = 1 + 0.85z^{-1}, \\ \boldsymbol{\theta} = [a_1, a_2, b_1, b_2, c_1, d_1]^{\mathrm{T}} = [-0.40, 0.44, -1.80, 0.24, -0.48, 0.85]^{\mathrm{T}}. \end{cases}$$

Here, $\{u(t)\}$ is taken as a persistent excitation signal sequence with zero mean and unit variance, and $\{v(t)\}$ as a white noise sequence with zero mean and variance σ^2. Applying the presented BJ-RGELS and BJ-LSI algorithms to estimate the parameters of this system, the BJ-RGELS estimates and BJ-LSI estimates ($k = 6$) and their errors are shown in Tables 1 and 2 with the different data lengths $t = L = 1000$ and 2000, and the BJ-LSI estimates and their errors versus k are shown in Tables 3 and 4 with $t = L = 1000$ and 2000, where $\delta := \|\hat{\boldsymbol{\theta}}(L) - \boldsymbol{\theta}\|/\|\boldsymbol{\theta}\|$ (the BJ-RGELS algorithm) or $\delta := \|\hat{\boldsymbol{\theta}}_k(L) - \boldsymbol{\theta}\|/\|\boldsymbol{\theta}\|$ (the BJ-LSI algorithm) is the parameter estimation errors. When the noise variance $\sigma^2 = 0.50^2$ and $\sigma^2 = 1.00^2$, the corresponding noise-to-signal ratios are $\delta_{\mathrm{ns}} = 44.80\%$ and $\delta_{\mathrm{ns}} = 89.60\%$, respectively.

When $L = 2000$, the BJ-LSI estimation errors δ versus k are shown in Fig. 4 with $\sigma^2 = 0.50^2$ and $\sigma^2 = 1.00^2$.

From the simulation results of Tables 1 to 4 and Fig. 4, we can draw the following conclusions.

- Under the same noise variance and data length, the parameter estimate errors of the BJ-LSI algorithm are smaller than those of the BJ-RGELS algorithm. In other words, the parameter estimates given by the BJ-LSI algorithm have higher accuracy than those given by the BJ-RGELS algorithm. For example, when $\sigma^2 = 0.50^2$ and $L = 1000$ and $L = 2000$, the

Table 1
The BJ-RGELS estimates $\hat{\boldsymbol{\theta}}(L)$ and errors.

σ^2	L	a_1	a_2	b_1	b_2	c_1	d_1	δ (%)
0.50^2	1000	−0.43560	0.45139	−1.77776	0.31880	−0.41786	0.87320	5.21067
	2000	−0.40965	0.45282	−1.78538	0.27109	−0.42151	0.88976	3.73980
1.00^2	1000	−0.55417	0.40760	−1.75827	0.52565	−0.39696	0.85491	15.80942
	2000	−0.42151	0.41491	−1.76851	0.29627	−0.40142	0.87382	5.10352
True values		−0.40000	0.44000	−1.80000	0.24000	−0.48000	0.85000	

Table 2
The BJ-LSI estimates $\hat{\boldsymbol{\theta}}_k(L)$ and errors ($k = 10$).

σ^2	L	a_1	a_2	b_1	b_2	c_1	d_1	δ (%)
0.50^2	1000	−0.41313	0.44727	−1.80013	0.25595	−0.47400	0.84678	1.06893
	2000	−0.40411	0.44112	−1.79450	0.25085	−0.46716	0.84906	0.84894
1.00^2	1000	−0.41938	0.45176	−1.79951	0.25895	−0.47538	0.84311	1.43022
	2000	−0.40416	0.43990	−1.78921	0.25393	−0.46784	0.84812	1.01995
True values		−0.40000	0.44000	−1.80000	0.24000	−0.48000	0.85000	

Table 3
The BJ-LSI estimates and errors versus iteration k ($L = 1000$).

σ^2	k	a_1	a_2	b_1	b_2	c_1	d_1	δ (%)
0.50^2	1	−0.36617	0.04456	−1.84649	0.02042	−0.32829	−0.01938	46.29437
	2	−0.01628	0.22573	−1.79994	−0.48211	0.31045	1.03025	54.58666
	3	−0.42139	0.45027	−1.80087	0.27122	−0.29887	0.82315	8.72722
	4	−0.43008	0.46280	−1.80701	0.28745	−0.46511	0.74858	5.55991
	5	−0.41834	0.44426	−1.79912	0.26181	−0.46565	0.78824	3.24606
	6	−0.40846	0.44298	−1.80344	0.24846	−0.47802	0.80713	2.08699
	7	−0.41203	0.44694	−1.79689	0.25269	−0.46972	0.82947	1.39060
	8	−0.41484	0.44663	−1.80222	0.26149	−0.47105	0.84360	1.36016
	9	−0.41388	0.44717	−1.79803	0.25819	−0.47587	0.84173	1.20098
	10	−0.41313	0.44727	−1.80013	0.25595	−0.47400	0.84678	1.06893
1.00^2	1	−0.62007	0.08168	−1.84138	0.45292	−0.61993	−0.06989	48.64874
	2	−0.56173	0.32673	−1.79916	0.50951	−0.24777	0.68571	20.45001
	3	−0.38445	0.41166	−1.80530	0.19116	−0.44129	0.72373	6.73778
	4	−0.39050	0.45587	−1.81118	0.20625	−0.48177	0.73975	5.46760
	5	−0.39391	0.43920	−1.79914	0.20528	−0.47468	0.79313	3.12859
	6	−0.40185	0.44573	−1.80624	0.23127	−0.47821	0.81711	1.63891
	7	−0.41615	0.45297	−1.79509	0.24865	−0.47531	0.82881	1.47300
	8	−0.42063	0.45040	−1.80301	0.26678	−0.47400	0.84187	1.71992
	9	−0.41913	0.45384	−1.79655	0.25769	−0.47858	0.83896	1.47818
	10	−0.41938	0.45176	−1.79951	0.25895	−0.47538	0.84311	1.43022
True values		−0.40000	0.44000	−1.80000	0.24000	−0.48000	0.85000	

estimation errors of the BJ-RGELS algorithm in the last column of Table 1 are 5.21067% and 3.73980%, respectively, while the estimation errors of the BJ-LSI algorithm in the last column of Table 2 are 1.06893% and 0.84894%, respectively.

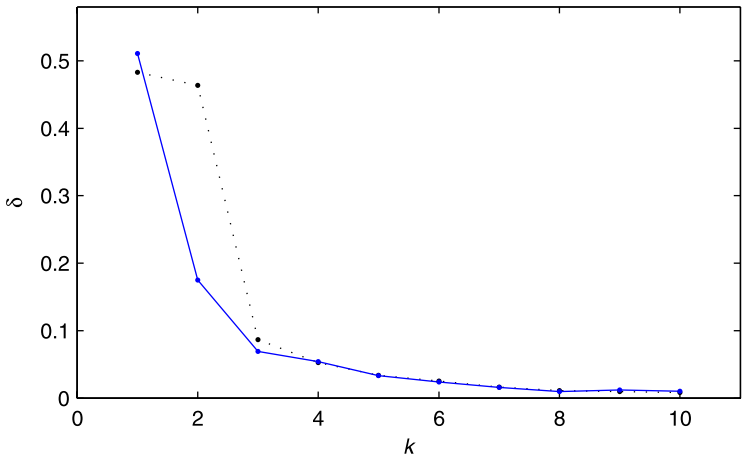
- As the iteration k increases, the parameter estimation errors of the BJ-LSI algorithm become small since the error curves in Fig. 4 decrease. The parameter estimation errors of the BJ-LSI algorithm are close to zero with the increasing of the data length L and iteration k.
- For the BJ-LSI algorithm, a lower noise level can lead to a faster convergence rate since the estimation errors with $\sigma^2 = 0.50^2$ are generally smaller than ones with $\sigma^2 = 1.00^2$ in the last columns of Tables 3 and 4.
- The parameter estimation errors of the BJ-LSI algorithm are small than those of the gradient based iterative algorithm for Box–Jenkins models (the BJ-GI algorithm for short) in [14] under the same iteration. That is, the BJ-LSI algorithm has a faster convergence rate than the BJ-GI algorithm.

5. Conclusions

This paper presents a recursive least squares and a least squares based iterative algorithms for estimating the parameters of Box–Jenkins models. The proposed BJ-LSI iterative algorithm can provide more accurate estimates of the system parameters than the recursive BJ-RGELS algorithm. This method can be extended to multiple-input single-output systems [15], multi-input multi-output systems [16], non-uniformly sampled systems [17].

Table 4
The BJ-LSI estimates and errors versus iteration k ($L = 2000$).

σ^2	k	a_1	a_2	b_1	b_2	c_1	d_1	δ (%)
0.50^2	1	−0.31242	0.04540	−1.82770	0.06446	−0.40339	−0.08434	48.29386
	2	−0.11398	0.23358	−1.79280	−0.28284	0.27481	0.99777	46.35699
	3	−0.40641	0.43892	−1.79300	0.25380	−0.29712	0.82243	8.65499
	4	−0.41912	0.45540	−1.79926	0.27845	−0.45925	0.74871	5.26701
	5	−0.40822	0.43911	−1.79549	0.25449	−0.46158	0.78247	3.35993
	6	−0.39940	0.43796	−1.79370	0.24124	−0.47487	0.79640	2.52912
	7	−0.40195	0.44150	−1.79202	0.24611	−0.46656	0.81939	1.63108
	8	−0.40282	0.44054	−1.79610	0.24986	−0.46935	0.83215	1.09581
	9	−0.40443	0.44185	−1.79456	0.25148	−0.46816	0.83956	0.97044
	10	−0.40411	0.44112	−1.79450	0.25085	−0.46716	0.84906	0.84894
1.00^2	1	−0.56819	0.08268	−1.81956	0.50409	−0.67806	−0.11782	51.09532
	2	−0.51117	0.33616	−1.78589	0.44011	−0.25856	0.68152	17.48798
	3	−0.37259	0.39977	−1.78716	0.19274	−0.43648	0.72630	6.90706
	4	−0.37665	0.44525	−1.79526	0.20680	−0.47490	0.74194	5.39515
	5	−0.38160	0.43188	−1.79236	0.20660	−0.46762	0.79254	3.30630
	6	−0.38759	0.43617	−1.78578	0.22388	−0.47609	0.80602	2.36727
	7	−0.39806	0.44276	−1.78533	0.24143	−0.46974	0.82175	1.56798
	8	−0.39808	0.43872	−1.79156	0.24529	−0.47040	0.83443	0.97648
	9	−0.40624	0.44459	−1.78936	0.25679	−0.46982	0.83887	1.21779
	10	−0.40416	0.43990	−1.78921	0.25393	−0.46784	0.84812	1.01995
True values		−0.40000	0.44000	−1.80000	0.24000	−0.48000	0.85000	



Solid line: $\sigma^2 = 0.50^2$, dotted line: $\sigma^2 = 1.00^2$

Fig. 4. The estimation errors δ versus k with $\sigma^2 = 0.50^2$ and $\sigma^2 = 1.00^2$ ($L = 2000$).

References

[1] G.H. Golub, C.F. Van Loan, Matrix Computations, 3rd ed., Johns Hopkins University Press, Baltimore, MD, 1996.
[2] F. Ding, T. Chen, Gradient based iterative algorithms for solving a class of matrix equations, IEEE Trans. Automat. Control 50 (8) (2005) 1216–1221.
[3] F. Ding, P.X. Liu, J. Ding, Iterative solutions of the generalized Sylvester matrix equations by using the hierarchical identification principle, Appl. Math. Comput. 197 (1) (2008) 41–50.
[4] F. Ding, T. Chen, Iterative least squares solutions of coupled Sylvester matrix equations, Syst. Control Lett. 54 (2) (2005) 95–107.
[5] A. Kilicman, Z.Al. Zhour, Vector least-squares solutions for coupled singular matrix equations, J. Comput. Appl. Math. 206 (2) (2007) 1051–1069.
[6] Z.Al. Zhour, A. Kilicman, Some new connections between matrix products for partitioned and non-partitioned matrices, Comput. Math. Appl. 54 (6) (2007) 763–784.
[7] G.C. Goodwin, K.S. Sin, Adaptive Filtering Prediction and Control, Prentice–Hall, Englewood Cliffs, NJ, 1984.
[8] L. Ljung, System Identification: Theory for the User, 2nd ed., Prentice–Hall, Englewood Cliffs, NJ, 1999.
[9] F. Ding, Y. Shi, T. Chen, Performance analysis of estimation algorithms of non-stationary ARMA processes, IEEE Trans. Signal Process. 54 (3) (2006) 1041–1053.
[10] K.S. Narendra, P.G. Gallman, An iterative method for the identification of nonlinear systems using a Hammerstein model, IEEE Trans. Automat. Control 11 (3) (1966) 546–550.
[11] E.W. Bai, D. Li, Convergence of the iterative Hammerstein system identification algorithm, IEEE Trans. Automat. Control 49 (11) (2004) 1929–1940.
[12] Y. Liu, E.W. Bai, Iterative identification of Hammerstein systems, Automatica 43 (2) (2007) 346–354.
[13] F. Ding, P.X. Liu, G. Liu, Gradient based and least-squares based iterative identification methods for OE and OEMA systems, Digital Signal Process. 20 (3) (2010) 664–677.

[14] D.Q. Wang, J.Y. Dai, F. Ding, Gradient-based iterative parameter estimation for Box–Jenkins systems with finite measurement data, in: 48th IEEE Conference on Decision and Control and 28th Chinese Control Conference, Shanghai, China, December 16–18, 2009, pp. 239–243.
[15] Y. J Liu, Y.S. Xiao, X.L. Zhao, Multi-innovation stochastic gradient algorithm for multiple-input single-output systems using the auxiliary model, Appl. Math. Comput. 215 (4) (2009) 1477–1483.
[16] L.L. Han, F. Ding, Multi-innovation stochastic gradient algorithms for multi-input multi-output systems, Digital Signal Process. 19 (4) (2009) 545–554.
[17] Y.J. Liu, L. Xie, F. Ding, An auxiliary model based recursive least squares parameter estimation algorithm for non-uniformly sampled multirate systems, J. Syst. Control Eng. 223 (4) (2009) 445–454.

Yanjun Liu was born in Jingjiang, Jiangsu Province, China. She received the B.Sc. degree from Jiangsu Teacher's University of Technology (Changzhou, China) in 2003, and the M.Sc. degree from the School of Communication and Control Engineering, Jiangnan University (Wuxi, China) in 2007. From 2003 to 2007, she was a teacher in the Jiangyin Huazi Vocational School, Jiangyin, Jiangsu Province. She is a Ph.D. candidate in the School of Communication and Control Engineering at the Jiangnan University. Her interests are system identification and parameter estimation.

Dongqing Wang was born in Shenyang, Liaoning Province, China. She received the B.Sc. and M.Sc. degrees from the Department of Electrical Engineering, Shandong University of Technology (Jinan, China) in 1986 and 1988, respectively, and the Ph.D. degree from the College of Automation Engineering, Tianjin University (Tianjin, China) in 2006. She is with the College of Automation Engineering, Qingdao University (Qingdao, China). Her research interests are stochastic systems, system identification, process modeling and control.

Feng Ding was born in Guangshui, Hubei Province. He received the B.Sc. degree from the Hubei University of Technology (Wuhan, China) in 1984, and the M.Sc. and Ph.D. degrees in automatic control both from the Department of Automation, Tsinghua University in 1991 and 1994, respectively.

From 1984 to 1988, he was an Electrical Engineer at the Hubei Pharmaceutical Factory, Xiangfan, China. From 1994 to 2002, he was with the Department of Automation, Tsinghua University, Beijing, China and he was a Post-Doctoral Fellow/Research Associate at the University of Alberta, Edmonton, Canada from 2002 to 2005.

He was a Visiting Professor in the Department of Systems and Computer Engineering, Carleton University, Ottawa, Canada from May to December 2008 and a Research Associate in the Department of Aerospace Engineering, Ryerson University, Toronto, Canada, from January to October 2009.

He has been a Professor in the School of Communication and Control Engineering, Jiangnan University, Wuxi, China since 2004. He is a Colleges and Universities "Blue Project" Middle-Aged Academic Leader (Jiangsu, China). His current research interests include model identification and adaptive control. He co-authored the book *Adaptive Control Systems* (Tsinghua University Press, Beijing, 2002), and published over 100 papers on modeling and identification as the first author.